Phase 4: 接口设计与工程思考

1. 项目结构总览

1.1 你需要工作的位置

```
assignment1-basics/
├── cs336_basics/                ← 你的实现代码放这里
│   ├── __init__.py              ← 已存在，只有版本号
│   ├── pretokenization_example.py  ← 已存在，分块工具
│   ├── model.py                 ← 你创建：所有模型组件
│   ├── nn_utils.py              ← 你创建：softmax, cross_entropy, gradient_clipping
│   ├── optimizer.py             ← 你创建：AdamW
│   ├── tokenizer.py             ← 你创建：BPE 训练 + Tokenizer 类
│   ├── data.py                  ← 你创建：get_batch
│   ├── training.py              ← 你创建：checkpoint, 训练循环
│   └── generation.py            ← 你创建：文本生成
├── tests/
│   ├── adapters.py              ← 你需要修改：连接你的实现到测试
│   ├── conftest.py              ← 不要修改
│   ├── common.py                ← 不要修改
│   ├── test_*.py                ← 不要修改
│   └── fixtures/                ← 测试数据，不要修改
└── pyproject.toml
```

1.2 测试固定参数（来自 conftest.py）

测试使用的固定参数（理解这些对调试很重要）：

fixture 名	值	说明
n_layers	3	层数
vocab_size	10,000	词表大小
batch_size	4	批大小
n_queries	12	query 序列长度

fixture 名	值	说明
n_keys	16	key 序列长度（也用作 max_seq_len）
n_heads	4	注意力头数
d_head	16	每个 head 的维度
d_model	n_heads × d_head = 64	模型维度
d_ff	128	FFN 中间维度
theta	10000.0	RoPE 参数

测试用的参考模型（`ts_state_dict`）从 `fixtures/ts_tests/model.pt` 加载，配置在 `model_config.json` 中。

2. Adapter 对接策略

2.1 adapter 的工作模式

每个 adapter 函数的通用模式：

```
1. 接收：维度参数 + 权重张量 + 输入张量
2. 实例化你的模块
3. 将传入的权重注入你的模块
4. 调用 forward
5. 返回输出
```

2.2 权重注入的两种方式

方式 A：手动赋值（推荐用于简单模块）

```
# 在 adapter 中
my_linear = MyLinear(d_in, d_out)
my_linear.weight.data = weights  # 直接覆盖 Parameter 的数据
return my_linear(in_features)
```

方式 B：load_state_dict（推荐用于复杂模块）

```
# 在 adapter 中
my_block = MyTransformerBlock(d_model, num_heads, d_ff, ...)
my_block.load_state_dict(weights)  # 需要你的 key 名与 weights dict 匹配
return my_block(in_features)
```

关键：方式 B 要求你的模块内部子模块命名与 adapter 传入的 weight dict keys 完全匹配。

3. 每个 Adapter 的具体接口分析

3.1 run_linear

输入：d_in, d_out, weights(d_out×d_in), in_features(...×d_in)
输出：(...×d_out)

你的 Linear 类需要：

- 存储参数名为 `weight`，形状 `(d_out, d_in)`
- `forward` 接收任意 batch 维度的输入

3.2 run_embedding

输入：vocab_size, d_model, weights(vocab_size×d_model), token_ids(...)
输出：(...×d_model)

你的 Embedding 类需要：

- 存储参数名为 `weight`，形状 `(vocab_size, d_model)`
- `forward` 接收任意形状的整数张量

3.3 run_rmsnorm

输入：d_model, eps, weights(d_model,), in_features(...×d_model)
输出：(...×d_model)

你的 RMSNorm 类需要：

- 存储参数名为 `weight`，形状 `(d_model,)`
- 初始化为全 1

3.4 run_silu

输入：in_features(任意形状)
输出：同形状

纯函数，不需要 nn.Module。可以是独立函数或 Module。

3.5 run_swiglu

输入：d_model, d_ff, w1_weight, w2_weight, w3_weight, in_features
权重形状：

- w1: (d_ff, d_model) — 门控分支
- w2: (d_model, d_ff) — 输出投影
- w3: (d_ff, d_model) — 信息分支

你的 SwiGLU 类内部子模块命名建议：

- `w1` — Linear(d_model, d_ff)，参数名 `w1.weight`
- `w2` — Linear(d_ff, d_model)，参数名 `w2.weight`
- `w3` — Linear(d_model, d_ff)，参数名 `w3.weight`

3.6 run_scaled_dot_product_attention

输入：Q(...×queries×d_k), K(...×keys×d_k), V(...×keys×d_v), mask(可选)
输出：(...×queries×d_v)

纯函数，不需要 nn.Module。

注意测试：

- `test_scaled_dot_product_attention`：3D 输入 (batch, seq, d)
- `test_4d_scaled_dot_product_attention`：4D 输入 (batch, heads, seq, d)
- mask 形状与 QK^T 匹配

3.7 run_multihead_self_attention

输入：d_model, num_heads, 四个投影权重, in_features
权重来源（从 state_dict 提取）：

- `layers.0.attn.q_proj.weight` — 形状 (d_model, d_model)
- `layers.0.attn.k_proj.weight` — 形状 (d_model, d_model)

- `layers.0.attn.v_proj.weight` — 形状 (d_model, d_model)
- `layers.0.attn.output_proj.weight` — 形状 (d_model, d_model)

你的 MHA 类内部命名建议：

- `q_proj` → Linear，参数 key: `q_proj.weight`
- `k_proj` → Linear，参数 key: `k_proj.weight`
- `v_proj` → Linear，参数 key: `v_proj.weight`
- `output_proj` → Linear，参数 key: `output_proj.weight`

注意： `run_multihead_self_attention` 不使用 RoPE，而 `run_multihead_self_attention_with_rope` 使用。

3.8 run_rope

输入： d_k, theta, max_seq_len, in_query_or_key(...×seq_len×d_k), token_positions(...×seq_len)
输出： 同形状

RoPE 模块的 forward 需要同时接受输入张量和位置张量。

3.9 run_transformer_block

输入： d_model, num_heads, d_ff, max_seq_len, theta, weights(dict), in_features
weights 的 key 名（很重要！）：

```
attn.q_proj.weight        (d_model, d_model)
attn.k_proj.weight        (d_model, d_model)
attn.v_proj.weight        (d_model, d_model)
attn.output_proj.weight   (d_model, d_model)
ln1.weight                (d_model,)
ln2.weight                (d_model,)
ffn.w1.weight             (d_ff, d_model)    ← 注意：文档写的是 (d_model, d_ff)，但看 adapt
ffn.w2.weight             (d_model, d_ff)    ← 看 adapter 中的 Shape 注释可能有出入
ffn.w3.weight             (d_ff, d_model)
```

重要： 测试代码做了 `k.replace("layers.0.", "")` 来提取单层的权重，所以你的 TransformerBlock 的 state_dict 必须匹配上面的 key。

你的 TransformerBlock 内部命名建议：

```
self.attn = MultiHeadSelfAttention(...)   → 产生 attn.q_proj.weight 等
self.ln1 = RMSNorm(...)                   → 产生 ln1.weight
self.ln2 = RMSNorm(...)                   → 产生 ln2.weight
self.ffn = SwiGLU(...)                    → 产生 ffn.w1.weight 等
```

3.10 run_transformer_lm

weights 的 key 名（最完整的）：

```
token_embeddings.weight                    (vocab_size, d_model)
layers.{i}.attn.q_proj.weight              (d_model, d_model)
layers.{i}.attn.k_proj.weight              (d_model, d_model)
layers.{i}.attn.v_proj.weight              (d_model, d_model)
layers.{i}.attn.output_proj.weight         (d_model, d_model)
layers.{i}.ln1.weight                      (d_model,)
layers.{i}.ln2.weight                      (d_model,)
layers.{i}.ffn.w1.weight                   (d_ff, d_model)
layers.{i}.ffn.w2.weight                   (d_model, d_ff)
layers.{i}.ffn.w3.weight                   (d_ff, d_model)
ln_final.weight                            (d_model,)
lm_head.weight                             (vocab_size, d_model)
```

你的 TransformerLM 内部命名建议：

```
self.token_embeddings = Embedding(vocab_size, d_model)
self.layers = nn.ModuleList([TransformerBlock(...) for _ in range(num_layers)])
self.ln_final = RMSNorm(d_model)
self.lm_head = Linear(d_model, vocab_size)
```

这样 `state_dict()` 自动生成的 key 就是：

- `token_embeddings.weight`
- `layers.0.attn.q_proj.weight`， `layers.1.attn.q_proj.weight`，…
- `ln_final.weight`
- `lm_head.weight`

与测试期望完全匹配。

4. BPE 相关 Adapter

4.1 run_train_bpe

输入：input_path, vocab_size, special_tokens
输出：(vocab: dict[int, bytes], merges: list[tuple[bytes, bytes]])

vocab 的 key 是 token ID (int)，value 是 token 对应的字节串 (bytes)。

测试要求：

- `test_train_bpe_speed` ：在 `corpus.en` 上训练 500 词表，耗时 < 1.5 秒
- `test_train_bpe` ：结果与参考 vocab/merges 精确匹配
- `test_train_bpe_special_tokens` ：special token 不被 merge

4.2 get_tokenizer

输入：vocab, merges, special_tokens
返回：一个 Tokenizer 对象，需要有 `encode` ， `decode` ， `encode_iterable` 方法

测试要求：

- 与 tiktoken 的 GPT-2 encoding 结果完全一致
- 支持 special tokens
- roundtrip: encode → decode 得回原文
- `encode_iterable` 接收 file-like 可迭代对象，返回 token ID 迭代器
- 内存限制测试： `encode_iterable` 处理 5MB 文件时额外内存 < 1MB

5. 其他 Adapter

5.1 run_softmax / run_cross_entropy

纯函数接口。测试会与 PyTorch 的 `F.softmax` 和 `F.cross_entropy` 对比。

cross_entropy 注意：测试传入 `inputs.view(-1, vocab_size)` 和 `targets.view(-1)` ，所以你的实现接收 2D logits + 1D targets。

5.2 run_gradient_clipping

原地修改 `parameters` 的 `.grad` 。测试与 `torch.nn.utils.clip_grad_norm_` 对比。

注意：测试中最后一个参数 `requires_grad_(False)` ，你的实现需要跳过 `grad is None` 的参数。

5.3 get_adamw_cls

返回你的 AdamW **类**（不是实例）。测试代码会这样用：

```
opt = get_adamw_cls()(model.parameters(), lr=..., weight_decay=..., betas=..., eps=...)
```

测试会运行 1000 步优化，然后与 PyTorch 的 `torch.optim.AdamW` 对比权重结果。

5.4 run_get_lr_cosine_schedule

纯函数。注意测试中 `it=0` 时 lr=0（第 0 步学习率为 0）。

5.5 run_get_batch

返回 `(inputs, labels)` 两个 LongTensor，形状 `(batch_size, context_length)` 。

测试要求：

- labels = inputs 右移一位
- 起始位置范围 `[0, len(dataset) - context_length - 1]`
- 均匀随机采样（统计检验）
- device 参数必须生效

5.6 run_save_checkpoint / run_load_checkpoint

测试用你自己的 AdamW 类创建优化器，保存后加载回来，检查模型参数和优化器状态是否一致。

6. State Dict Key 命名总结（核心参考表）

这是你在设计模块命名时最重要的参考。如果命名不匹配， `load_state_dict` 会报错。

```
TransformerLM
├── token_embeddings          → Embedding    → .weight (vocab_size, d_model)
├── layers                    → ModuleList
│   └── [i]                   → TransformerBlock
│       ├── ln1               → RMSNorm      → .weight (d_model,)
│       ├── attn              → MHA
│       │   ├── q_proj        → Linear       → .weight (d_model, d_model)
│       │   ├── k_proj        → Linear       → .weight (d_model, d_model)
│       │   ├── v_proj        → Linear       → .weight (d_model, d_model)
│       │   └── output_proj   → Linear       → .weight (d_model, d_model)
│       ├── ln2               → RMSNorm      → .weight (d_model,)
│       └── ffn               → SwiGLU
│           ├── w1            → Linear       → .weight (d_ff, d_model)
│           ├── w2            → Linear       → .weight (d_model, d_ff)
│           └── w3            → Linear       → .weight (d_ff, d_model)
├── ln_final                  → RMSNorm      → .weight (d_model,)
└── lm_head                   → Linear       → .weight (vocab_size, d_model)
```

7. 工程建议

7.1 渐进式开发策略

每实现一个模块，立即跑对应测试：

```
uv run pytest -k test_linear -v
uv run pytest -k test_embedding -v
uv run pytest -k test_rmsnorm -v
# ... 逐个推进
```

7.2 调试 state_dict 不匹配

如果 `load_state_dict` 报错 "unexpected key" 或 "missing key"：

```
# 在 adapter 中临时打印你的模块 key:
print(my_module.state_dict().keys())
# 对比传入的 weights.keys()
```

7.3 数值精度

- 测试用的 tolerance：大多数是 `atol=1e-6` ，整体模型是 `atol=1e-4, rtol=1e-2`
- 注意 RMSNorm 的 float32 upcast
- 注意 cross_entropy 的 log-sum-exp trick
- AdamW 有两种等价写法，测试先对比 PyTorch 结果，不匹配时对比参考快照

7.4 Tokenizer 类的接口契约

Tokenizer 对象必须暴露：

- `.encode(text: str) -> list[int]`
- `.decode(ids: list[int]) -> str`
- `.encode_iterable(iterable: Iterable[str]) -> Iterator[int]`

`encode_iterable` 接收 file 对象（ `open(path)` 返回的），逐行读入。